THE DATA ENGINEER LAB

DATABRICKS FROM ZERO TO HERO

A 30-Day Interview & Career Readiness Series

DAY 10

Schema Enforcement vs Schema Evolution

SECTION 01

Schema Enforcement — Protecting the Table

Schema enforcement, also called schema validation on write, is Delta Lake's default protective behavior. Every new write is checked against the target table's schema before it is allowed to commit.

CORE CONCEPT

Delta Lake uses schema validation on write, meaning all new writes to a table are checked for compatibility with the target table's schema at write time. If the schema is not compatible, Delta Lake cancels the transaction and raises an exception, rather than silently writing mismatched data. It is impossible to bypass this enforcement.

What Gets Rejected by Default

- Writes with columns that don't exist in the target table's schema.
- Writes with a different column ordering than what the table expects, when types don't align.
- Writes with mismatched data types for existing columns, such as a string arriving where an integer is expected.

```
# This raises an AnalysisException if 'new_column' doesn't exist
# in the target table and mergeSchema is not set
new_data.write.format("delta").mode("append").save("/path/to/table")

# Error:
# AnalysisException: A schema mismatch detected when writing
# to the Delta table...
```

SECTION 02

Schema Evolution — Controlled, Opt-In Change

Schema evolution is the mechanism that lets you intentionally change a table's schema, most commonly to add new columns, without needing to rewrite the entire table.

CORE CONCEPT

Schema evolution is activated by adding the mergeSchema option to a write or writeStream command. Any columns present in the incoming data but not in the target table are automatically added to the end of the schema as part of that write transaction, and existing rows are automatically backfilled with NULL for the new column.

```
# Enable schema evolution for a single write
new_data.write.format("delta") \
    .mode("append") \
    .option("mergeSchema", "true") \
    .save("/path/to/table")

# Or enable it session-wide for all writes (use with caution)
```

```
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
```

USE WITH CAUTION

Session-wide auto-merge means schema enforcement will no longer warn you about unintended schema mismatches for any write in that session. Most teams prefer scoping mergeSchema to individual writes rather than enabling it globally.

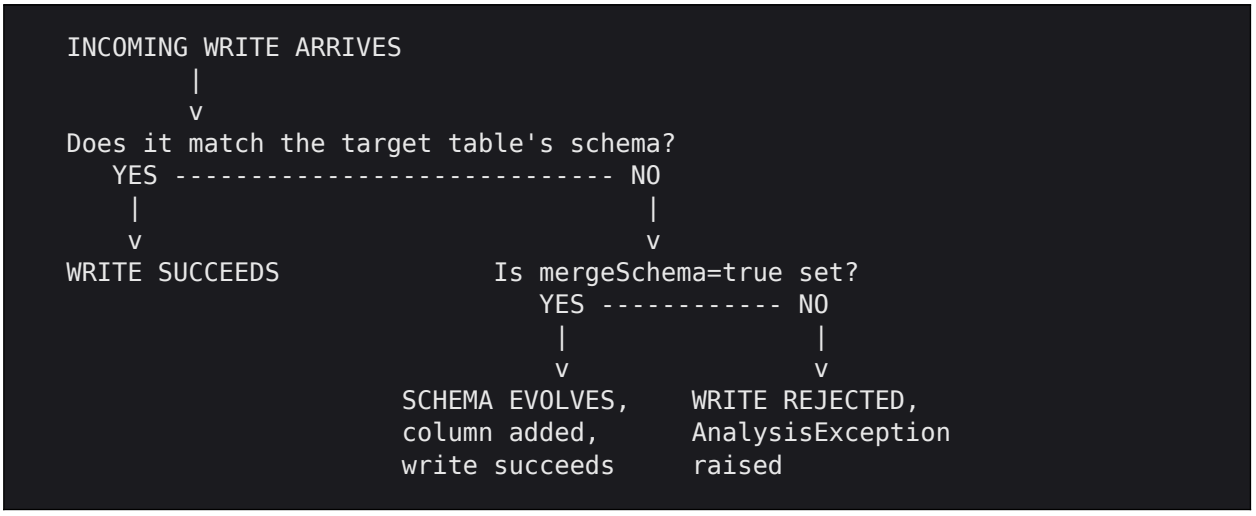
SECTION 03

Side-by-Side Comparison

Aspect	Schema Enforcement	Schema Evolution
Default Behavior	On by default, cannot be disabled	Off by default, must be explicitly enabled
Purpose	Protects the table from mismatched or garbage data	Allows controlled, intentional schema changes
Trigger	Automatic on every write	Requires mergeSchema option or autoMerge config
Typical Outcome	Rejects the write with an exception on mismatch	Adds new columns, backfills existing rows with NULL
Risk if Misused	None, it is a safety feature	Can mask genuine data quality issues if overused

SECTION 04

How a Write Is Evaluated



REMEMBER THIS FOR INTERVIEWS

Schema enforcement is the default gatekeeper; schema evolution is the explicit key that lets specific, intended changes through that gate. They are not opposites in the sense of competing features, they work together, enforcement is always active, and evolution is the sanctioned exception path.

SECTION 05

Why This Is Better Than Plain Parquet

Plain Parquet tables handle schema very differently, and this contrast is a strong interview talking point.

- Parquet tables without Hive metastore information have no built-in schema enforcement at all.
- Even Parquet tables registered in a metastore only enforce schema if accessed by table name; accessing the underlying path directly bypasses enforcement entirely.
- The mergeSchema option is silently ignored when writing to a plain Parquet table, so there is no safe, built-in schema evolution path.
- Delta Lake determines a table's schema by reading the transaction log, not by opening and inspecting every individual Parquet file, which makes it both faster and more reliable than schema-on-read approaches.

SECTION 06

Glossary — Day 10 Terms

Term	Definition
Schema Enforcement	Delta Lake's default behavior of validating every write against the target table's schema and rejecting mismatches.
Schema Evolution	The opt-in mechanism, enabled via mergeSchema, that allows a table's schema to change in controlled ways, typically by adding columns.
mergeSchema	The write option that enables schema evolution for a single write operation.
autoMerge	A session-wide Spark configuration that enables schema evolution for all writes in that session.
Schema on Write	Delta Lake's model where the schema is validated and defined at write time, rather than inferred at read time.
overwriteSchema	An option used for non-additive schema changes, which rewrites all Parquet files associated with the table.

SECTION 07

Interview Questions & Answers

Q1. What is the difference between schema enforcement and schema evolution?

A. Schema enforcement is Delta Lake's default, always-on behavior that rejects writes with a mismatched schema. Schema evolution is an opt-in mechanism, enabled with mergeSchema, that allows a table's schema to change in controlled ways, such as adding new columns.

Q2. Can schema enforcement be disabled in Delta Lake?

A. No. It is impossible to bypass Delta Lake's schema enforcement; it is always active unless a specific write explicitly opts into schema evolution using mergeSchema.

Q3. What happens to existing rows when a new column is added through schema evolution?

A. Delta Lake automatically updates the table schema to include the new column, and existing rows are automatically backfilled with NULL for that column.

Q4. What is the risk of enabling autoMerge session-wide instead of using mergeSchema per write?

A. Session-wide auto-merge means schema enforcement will no longer warn about unintended schema mismatches for any write in that session, which increases the risk of accidental schema drift going unnoticed.

Q5. How does Delta Lake determine a table's current schema so efficiently?

A. It works out the final schema by querying the transaction log, rather than opening and inspecting every individual Parquet file, which makes schema resolution both fast and reliable.

Q6. Why does plain Parquet not offer the same schema safety as Delta Lake?

A. Parquet tables without Hive metastore information have no built-in schema enforcement, enforcement can be bypassed by accessing the file path directly instead of the table name, and the mergeSchema option is silently ignored when writing to plain Parquet tables.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

Piyush Varma | The Data Engineer Lab